Solving Asymptotic Recurrences

Dr. Bugra Caskurlu[1]

[1] School of Computing
New Uzbekistan University

General Info

Recurrences

- When an algorithm contains a recursive call to itself, its running time can often be described by a **recurrence**.

General Info

Recurrences

- When an algorithm contains a recursive call to itself, its running time can often be described by a **recurrence**.
- A *recurrence is an equation or inequality* that describes a function in terms of **its value on smaller inputs**.

General Info

Recurrences

- When an algorithm contains a recursive call to itself, its running time can often be described by a **recurrence**.
- A *recurrence is an equation or inequality* that describes a function in terms of **its value on smaller inputs**.
- **Example:** The running-time of Merge Sort can be described by the recurrence:

General Info

Recurrences

- When an algorithm contains a recursive call to itself, its running time can often be described by a **recurrence**.
- A *recurrence is an equation or inequality* that describes a function in terms of **its value on smaller inputs**.
- **Example:** The running-time of Merge Sort can be described by the recurrence:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ 2 \cdot T(\frac{n}{2}) + \Theta(n) & \text{if } n > 1 \end{cases}$$

General Info

Recurrences

- When an algorithm contains a recursive call to itself, its running time can often be described by a **recurrence**.
- A *recurrence is an equation or inequality* that describes a function in terms of **its value on smaller inputs**.
- **Example:** The running-time of Merge Sort can be described by the recurrence:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ 2 \cdot T(\frac{n}{2}) + \Theta(n) & \text{if } n > 1 \end{cases}$$

- This week, we will see three methods for solving (obtaining Θ or O bounds) recurrences.

Solving Recurrences

Methods

- **Substitution Method:** We guess a bound and then use mathematical induction to prove our guess correct.

Solving Recurrences

Methods

- **Substitution Method:** We guess a bound and then use mathematical induction to prove our guess correct.
- **Recursion-Tree Method:** We convert the recurrence into a tree whose nodes represent the cost incurred at various levels of the recursion; we then *use techniques for bounding summations* to solve the recurrences.

Solving Recurrences

Methods

- **Substitution Method:** We guess a bound and then use mathematical induction to prove our guess correct.
- **Recursion-Tree Method:** We convert the recurrence into a tree whose nodes represent the cost incurred at various levels of the recursion; we then *use techniques for bounding summations* to solve the recurrences.
- **Master Method:** The master theorem provides bounds for recurrences of the form $T(n) = a \cdot T(\frac{n}{b}) + f(n)$, where $a \geq 1$, $b > 1$, and f(n) is a given function. It requires memorization of three cases, but once you do that, determining asymptotic bounds for **many**(but not all) recurrences is easy.

Technical Issues

Technicalities

- In practice, we neglect certain technical details when we state and solve recurrences.

Technical Issues

Technicalities

- In practice, we neglect certain technical details when we state and solve recurrences.
- **Example 1:** *Assumption of integer arguments to functions.* The recurrence for Merge-Sort is indeed $T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + \Theta(n)$.

Technical Issues

Technicalities

- In practice, we neglect certain technical details when we state and solve recurrences.
- **Example** 1**:** *Assumption of integer arguments to functions*. The recurrence for Merge-Sort is indeed $T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + \Theta(n)$.
- **Example** 2**:** *Boundary Conditions*. Since the running-time of an algorithm on a constant-sized input is a constant, the recurrences that arise from the running times of algorithms generally have $T(n) = \Theta(1)$ for sufficiently small n. Thus, we generally omit statements of the boundary conditions of recurrences and assume $T(n)$ is constant for small n.

Substitution Method

Steps and Usage

The substitution method for solving recurrences entails two steps:

Substitution Method

Steps and Usage

The substitution method for solving recurrences entails two steps:

- Guess the form of the solution.
- Use *mathematical induction* **correctly** to find the constants and show that the solution works.

Substitution Method

Steps and Usage

The substitution method for solving recurrences entails two steps:

- Guess the form of the solution.
- Use *mathematical induction* **correctly** to find the constants and show that the solution works.

The name comes from the substitution of the guessed answer for the function when the inductive hypothesis is applied to smaller values.

Substitution Method

Steps and Usage

The substitution method for solving recurrences entails two steps:

- Guess the form of the solution.
- Use *mathematical induction* **correctly** to find the constants and show that the solution works.

The name comes from the substitution of the guessed answer for the function when the inductive hypothesis is applied to smaller values.

- This method is powerful, but it obviously can be applied in cases when it is easy to guess the form of the answer.
- It can be used to establish either upper or lower bounds on a recurrence.

False Usage of Induction

Example

Find an asymptotic upper-bound for the recurrence: $T(n) = 2 \cdot T(\lfloor \frac{n}{2} \rfloor) + n$.

False Usage of Induction

Example

Find an asymptotic upper-bound for the recurrence: $T(n) = 2 \cdot T(\lfloor \frac{n}{2} \rfloor) + n$.

Pitfall

- In the substitution method, we should first make a guess for the solution, and let's guess that $T(n) = O(n)$.

False Usage of Induction

Example

Find an asymptotic upper-bound for the recurrence: $T(n) = 2 \cdot T(\lfloor \frac{n}{2} \rfloor) + n$.

Pitfall

- In the substitution method, we should first make a guess for the solution, and let's guess that $T(n) = O(n)$.
- We need to show that $T(n)$ is bounded by a constant multiple of n, and we will assume this property holds for $\lfloor \frac{n}{2} \rfloor$, i.e., $T(\lfloor \frac{n}{2} \rfloor) \leq c \cdot \lfloor \frac{n}{2} \rfloor$.

False Usage of Induction

Example

Find an asymptotic upper-bound for the recurrence: $T(n) = 2 \cdot T(\lfloor \frac{n}{2} \rfloor) + n$.

Pitfall

- In the substitution method, we should first make a guess for the solution, and let's guess that $T(n) = O(n)$.
- We need to show that $T(n)$ is bounded by a constant multiple of n, and we will assume this property holds for $\lfloor \frac{n}{2} \rfloor$, i.e., $T(\lfloor \frac{n}{2} \rfloor) \leq c \cdot \lfloor \frac{n}{2} \rfloor$.

$$
\begin{aligned}
T(n) &= 2 \cdot T(\lfloor \frac{n}{2} \rfloor) + n && (1)\\
&\leq c \cdot n + n && (2)\\
&= O(n). && (3)
\end{aligned}
$$

False Usage of Induction

Example

Find an asymptotic upper-bound for the recurrence: $T(n) = 2 \cdot T(\lfloor \frac{n}{2} \rfloor) + n$.

Pitfall

- In the substitution method, we should first make a guess for the solution, and let's guess that $T(n) = O(n)$.
- We need to show that $T(n)$ is bounded by a constant multiple of n, and we will assume this property holds for $\lfloor \frac{n}{2} \rfloor$, i.e., $T(\lfloor \frac{n}{2} \rfloor) \leq c \cdot \lfloor \frac{n}{2} \rfloor$.

$$
\begin{aligned}
T(n) &= 2 \cdot T(\lfloor \frac{n}{2} \rfloor) + n && (1)\\
&\leq c \cdot n + n && (2)\\
&= O(n). && (3)
\end{aligned}
$$

- **Error:** We haven't proved the exact form of the hypothesis, that is, $T(n) = c \cdot n$.

Correction

Reprove

- Let us present a valid proof for upper-bounding the recurrence $T(n) = 2 \cdot T(\lfloor \frac{n}{2} \rfloor) + n$ by the substitution method.

Correction

Reprove

- Let us present a valid proof for upper-bounding the recurrence $T(n) = 2 \cdot T(\lfloor \frac{n}{2} \rfloor) + n$ by the substitution method.
- We, again, need to make a guess for an upper-bound. Let's guess $T(n) = O(n \cdot \lg n)$.

Correction

Reprove

- Let us present a valid proof for upper-bounding the recurrence $T(n) = 2 \cdot T(\lfloor \frac{n}{2} \rfloor) + n$ by the substitution method.
- We, again, need to make a guess for an upper-bound. Let's guess $T(n) = O(n \cdot \lg n)$.
- We need to prove that $T(n) \leq c \cdot n \cdot \lg n$ for an appropriate choice of the constant $c > 0$.

Correction

Reprove

- Let us present a valid proof for upper-bounding the recurrence $T(n) = 2 \cdot T(\lfloor \frac{n}{2} \rfloor) + n$ by the substitution method.
- We, again, need to make a guess for an upper-bound. Let's guess $T(n) = O(n \cdot \lg n)$.
- We need to prove that $T(n) \leq c \cdot n \cdot \lg n$ for an appropriate choice of the constant $c > 0$.
- We assume that this bound holds for $\lfloor \frac{n}{2} \rfloor$, that is, $T(\lfloor \frac{n}{2} \rfloor) \leq c \cdot \lfloor \frac{n}{2} \rfloor \lg(\lfloor \frac{n}{2} \rfloor)$.

Correction

Reprove

- Let us present a valid proof for upper-bounding the recurrence $T(n) = 2 \cdot T(\lfloor \frac{n}{2} \rfloor) + n$ by the substitution method.
- We, again, need to make a guess for an upper-bound. Let's guess $T(n) = O(n \cdot \lg n)$.
- We need to prove that $T(n) \leq c \cdot n \cdot \lg n$ for an appropriate choice of the constant $c > 0$.
- We assume that this bound holds for $\lfloor \frac{n}{2} \rfloor$, that is, $T(\lfloor \frac{n}{2} \rfloor) \leq c \cdot \lfloor \frac{n}{2} \rfloor \lg(\lfloor \frac{n}{2} \rfloor)$.
- Substituting into the recurrence yields:

Correction

Reprove

- Let us present a valid proof for upper-bounding the recurrence $T(n) = 2 \cdot T(\lfloor \frac{n}{2} \rfloor) + n$ by the substitution method.
- We, again, need to make a guess for an upper-bound. Let's guess $T(n) = O(n \cdot \lg n)$.
- We need to prove that $T(n) \leq c \cdot n \cdot \lg n$ for an appropriate choice of the constant $c > 0$.
- We assume that this bound holds for $\lfloor \frac{n}{2} \rfloor$, that is, $T(\lfloor \frac{n}{2} \rfloor) \leq c \cdot \lfloor \frac{n}{2} \rfloor \lg(\lfloor \frac{n}{2} \rfloor)$.
- Substituting into the recurrence yields:

$$
\begin{aligned}
T(n) &\leq 2\left(c \cdot \lfloor \frac{n}{2} \rfloor \lg(\lfloor \frac{n}{2} \rfloor)\right) + n && (4)\\
&\leq c \cdot n \cdot \lg(\frac{n}{2}) + n && (5)\\
&= c \cdot n \cdot \lg n - c \cdot n + n && (6)\\
&\leq c \cdot n \cdot \lg n, \text{ for } c \geq 1. && (7)
\end{aligned}
$$

Technicalities

Base Case

Reconsider the recurrence:

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ 2 \cdot T(\lfloor \frac{n}{2} \rfloor) + n & \text{if } n > 1 \end{cases}$$

Technicalities

Base Case

Reconsider the recurrence:

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ 2 \cdot T(\lfloor \frac{n}{2} \rfloor) + n & \text{if } n > 1 \end{cases}$$

Technicalities

Base Case

Reconsider the recurrence:

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ 2 \cdot T(\lfloor \frac{n}{2} \rfloor) + n & \text{if } n > 1 \end{cases}$$

Details

- We proved that $T(n) \leq c \cdot n \cdot \lg n$ by using mathematical induction, right?

Technicalities

Base Case

Reconsider the recurrence:

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ 2 \cdot T(\lfloor \frac{n}{2} \rfloor) + n & \text{if } n > 1 \end{cases}$$

Details

- We proved that $T(n) \leq c \cdot n \cdot \lg n$ by using mathematical induction, right?
- Not quite. We also need to show that our solution holds for the boundary conditions (base cases).

Technicalities

Base Case

Reconsider the recurrence:

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ 2 \cdot T(\lfloor \frac{n}{2} \rfloor) + n & \text{if } n > 1 \end{cases}$$

Details

- We proved that $T(n) \leq c \cdot n \cdot \lg n$ by using mathematical induction, right?
- Not quite. We also need to show that our solution holds for the boundary conditions (base cases).
- Our hypothesis says that $T(1) \leq c \cdot 1 \cdot \lg 1 = 0$. But we have $T(n) = 1$!

Technicalities

Base Case

Reconsider the recurrence:

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ 2 \cdot T(\lfloor \frac{n}{2} \rfloor) + n & \text{if } n > 1 \end{cases}$$

Details

- We proved that $T(n) \leq c \cdot n \cdot \lg n$ by using mathematical induction, right?
- Not quite. We also need to show that our solution holds for the boundary conditions (base cases).
- Our hypothesis says that $T(1) \leq c \cdot 1 \cdot \lg 1 = 0$. But we have $T(n) = 1$!
- Asymptotic notation only requires us to prove that $T(n) \leq c \cdot n \cdot \lg n$ for $n \geq n_0$, where n_0 is a constant or our choosing. We can remove the difficult boundary condition from consideration in the inductive proof, and replace $T(1)$ by $T(2)$ and $T(3)$ as base cases.

Subtlety 1

Making a Good Guess

There is no general way to guess the correct solutions to recurrences. It requires experience and creativity. But there are some heuristics:

Subtlety 1

Making a Good Guess

There is no general way to guess the correct solutions to recurrences. It requires experience and creativity. But there are some heuristics:

- **Technique** 1**:** If a recurrence is similar to one you have seen before, then guessing a similar solution is reasonable.

Subtlety 1

Making a Good Guess

There is no general way to guess the correct solutions to recurrences. It requires experience and creativity. But there are some heuristics:

- **Technique 1:** If a recurrence is similar to one you have seen before, then guessing a similar solution is reasonable.
- **Example:** $T(n) = 2T(\lfloor \frac{n}{2} \rfloor + 17) + n$.

Subtlety 1

Making a Good Guess

There is no general way to guess the correct solutions to recurrences. It requires experience and creativity. But there are some heuristics:

- **Technique 1:** If a recurrence is similar to one you have seen before, then guessing a similar solution is reasonable.
- **Example:** $T(n) = 2T(\lfloor \frac{n}{2} \rfloor + 17) + n$. $T(n) = O(n \cdot \lg n)$.

Subtlety 1

Making a Good Guess

There is no general way to guess the correct solutions to recurrences. It requires experience and creativity. But there are some heuristics:

- **Technique 1:** If a recurrence is similar to one you have seen before, then guessing a similar solution is reasonable.
- **Example:** $T(n) = 2T(\lfloor \frac{n}{2} \rfloor + 17) + n$. $T(n) = O(n \cdot \lg n)$.
- **Technique 2:** Prove loose upper and lower bounds on the recurrence and then reduce the range of uncertainty.

Subtlety 1

Making a Good Guess

There is no general way to guess the correct solutions to recurrences. It requires experience and creativity. But there are some heuristics:

- **Technique 1:** If a recurrence is similar to one you have seen before, then guessing a similar solution is reasonable.
- **Example:** $T(n) = 2T(\lfloor \frac{n}{2} \rfloor + 17) + n$. $T(n) = O(n \cdot \lg n)$.
- **Technique 2:** Prove loose upper and lower bounds on the recurrence and then reduce the range of uncertainty.
- **Explanation:** For the above recurrence, we can start with a lower bound of $T(n) = \Omega(n)$ since we have the term n in the recurrence. And we can prove and initial bound upper bound of $T(n) = O(n^2)$.

Subtlety 1

Making a Good Guess

There is no general way to guess the correct solutions to recurrences. It requires experience and creativity. But there are some heuristics:

- **Technique 1:** If a recurrence is similar to one you have seen before, then guessing a similar solution is reasonable.
- **Example:** $T(n) = 2T(\lfloor \frac{n}{2} \rfloor + 17) + n$. $T(n) = O(n \cdot \lg n)$.
- **Technique 2:** Prove loose upper and lower bounds on the recurrence and then reduce the range of uncertainty.
- **Explanation:** For the above recurrence, we can start with a lower bound of $T(n) = \Omega(n)$ since we have the term n in the recurrence. And we can prove and initial bound upper bound of $T(n) = O(n^2)$.Then we can gradually lower the upper bound and raise the lower bound until we converge on the correct, asymptotically tight solution of $T(n) = \Theta(n \cdot \lg n)$.

Subtlety 2

Stronger Assumption

Consider the recurrence $T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + 1$.

Subtlety 2

Stronger Assumption

Consider the recurrence $T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + 1$. **Guess:** $T(n) = O(n)$, and try to show that $T(n) \leq c \cdot n$ for an appropriate choice of the constant c. Substituting our guess in the recurrence, we obtain:

Subtlety 2

Stronger Assumption

Consider the recurrence $T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + 1$. **Guess:** $T(n) = O(n)$, and try to show that $T(n) \leq c \cdot n$ for an appropriate choice of the constant c. Substituting our guess in the recurrence, we obtain:

$$T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + 1 \tag{8}$$

$$\leq c \cdot \lfloor \frac{n}{2} \rfloor + c \cdot \lceil \frac{n}{2} \rceil + 1 \tag{9}$$

$$= c \cdot n + 1. \tag{10}$$

Subtlety 2

Stronger Assumption

Consider the recurrence $T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + 1$. **Guess:** $T(n) = O(n)$, and try to show that $T(n) \leq c \cdot n$ for an appropriate choice of the constant c. Substituting our guess in the recurrence, we obtain:

$$T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + 1 \tag{8}$$

$$\leq c \cdot \lfloor \frac{n}{2} \rfloor + c \cdot \lceil \frac{n}{2} \rceil + 1 \tag{9}$$

$$= c \cdot n + 1. \tag{10}$$

Stronger Inductive Hypothesis

- We could not prove that $T(n) \leq c \cdot n$ for any choice of c. What is wrong here?

Subtlety 2

Stronger Assumption

Consider the recurrence $T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + 1$. **Guess:** $T(n) = O(n)$, and try to show that $T(n) \leq c \cdot n$ for an appropriate choice of the constant c. Substituting our guess in the recurrence, we obtain:

$$T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + 1 \tag{8}$$

$$\leq c \cdot \lfloor \frac{n}{2} \rfloor + c \cdot \lceil \frac{n}{2} \rceil + 1 \tag{9}$$

$$= c \cdot n + 1. \tag{10}$$

Stronger Inductive Hypothesis

- We could not prove that $T(n) \leq c \cdot n$ for any choice of c. What is wrong here?
- **Remedy:** *Make inductive hypothesis stronger by subtracting a lower order term*, i.e., our new guess is $T(n) = c \cdot n - b$.

Subtlety 2

Stronger Assumption

Consider the recurrence $T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + 1$. **Guess:** $T(n) = O(n)$, and try to show that $T(n) \leq c \cdot n$ for an appropriate choice of the constant c. Substituting our guess in the recurrence, we obtain:

$$T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + 1 \tag{8}$$

$$\leq c \cdot \lfloor \frac{n}{2} \rfloor + c \cdot \lceil \frac{n}{2} \rceil + 1 \tag{9}$$

$$= c \cdot n + 1. \tag{10}$$

Stronger Inductive Hypothesis

- We could not prove that $T(n) \leq c \cdot n$ for any choice of c. What is wrong here?
- **Remedy:** *Make inductive hypothesis stronger by subtracting a lower order term*, i.e., our new guess is $T(n) = c \cdot n - b$.
- Then we can show $T(n) \leq c \cdot n - 2 \cdot b + 1$, and this gives the desired result for $b \geq 1$.

Subtlety 2

Stronger Assumption

Consider the recurrence $T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + 1$. **Guess:** $T(n) = O(n)$, and try to show that $T(n) \leq c \cdot n$ for an appropriate choice of the constant c. Substituting our guess in the recurrence, we obtain:

$$T(n) = T(\lfloor \frac{n}{2} \rfloor) + T(\lceil \frac{n}{2} \rceil) + 1 \tag{8}$$

$$\leq c \cdot \lfloor \frac{n}{2} \rfloor + c \cdot \lceil \frac{n}{2} \rceil + 1 \tag{9}$$

$$= c \cdot n + 1. \tag{10}$$

Stronger Inductive Hypothesis

- We could not prove that $T(n) \leq c \cdot n$ for any choice of c. What is wrong here?
- **Remedy:** *Make inductive hypothesis stronger by subtracting a lower order term*, i.e., our new guess is $T(n) = c \cdot n - b$.
- Then we can show $T(n) \leq c \cdot n - 2 \cdot b + 1$, and this gives the desired result for $b \geq 1$.
- **Catch:** ***Positive constant c should be large enough to handle the boundary condition***.

Subtlety 3

Changing Variables

- Sometimes, a little algebraic manipulation can make an unknown recurrence similar to one you have seen.

Subtlety 3

Changing Variables

- Sometimes, a little algebraic manipulation can make an unknown recurrence similar to one you have seen.
- **Example:** $T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \lg n$, which looks different.

Subtlety 3

Changing Variables

- Sometimes, a little algebraic manipulation can make an unknown recurrence similar to one you have seen.
- **Example:** $T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \lg n$, which looks different.
- Setting $m = \lg n$ yields $T(2^m) = 2T(2^{\frac{m}{2}}) + m$.

Subtlety 3

Changing Variables

- Sometimes, a little algebraic manipulation can make an unknown recurrence similar to one you have seen.
- **Example:** $T(n) = 2T(\lfloor \sqrt{n} \rfloor) + \lg n$, which looks different.
- Setting $m = \lg n$ yields $T(2^m) = 2T(2^{\frac{m}{2}}) + m$.
- We can rename $S(m) = T(2^m)$ to produce $S(m) = 2S(\frac{m}{2}) + m$, which is very similar to our usual recurrence.

Subtlety 3

Changing Variables

- Sometimes, a little algebraic manipulation can make an unknown recurrence similar to one you have seen.
- **Example:** $T(n) = 2T(\lfloor\sqrt{n}\rfloor) + \lg n$, which looks different.
- Setting $m = \lg n$ yields $T(2^m) = 2T(2^{\frac{m}{2}}) + m$.
- We can rename $S(m) = T(2^m)$ to produce $S(m) = 2S(\frac{m}{2}) + m$, which is very similar to our usual recurrence.
- Thus, we have $S(m) = O(m \cdot \lg m)$. But we want a solution for $T(n)$!

Subtlety 3

Changing Variables

- Sometimes, a little algebraic manipulation can make an unknown recurrence similar to one you have seen.
- **Example:** $T(n) = 2T(\lfloor\sqrt{n}\rfloor) + \lg n$, which looks different.
- Setting $m = \lg n$ yields $T(2^m) = 2T(2^{\frac{m}{2}}) + m$.
- We can rename $S(m) = T(2^m)$ to produce $S(m) = 2S(\frac{m}{2}) + m$, which is very similar to our usual recurrence.
- Thus, we have $S(m) = O(m \cdot \lg m)$. But we want a solution for $T(n)$!
- $T(n) = T(2^m) = S(m) = O(m \cdot \lg m) = O(\lg n \cdot \lg \lg n)$.

Recursion Tree Method

Explanation

- In this method, we draw a recursion tree as we did in our analysis of Merge-Sort algorithm.

Recursion Tree Method

Explanation

- In this method, we draw a recursion tree as we did in our analysis of Merge-Sort algorithm.
- In a **recursion tree**, each node represents the cost of a single subproblem somewhere in the set of recursive function invocations.

Recursion Tree Method

Explanation

- In this method, we draw a recursion tree as we did in our analysis of Merge-Sort algorithm.
- In a **recursion tree**, each node represents the cost of a single subproblem somewhere in the set of recursive function invocations.
- We sum the costs in each level of the tree to obtain a set of per-level costs, and then we sum all the per-level costs to determine the total cost of all levels of the recursion.

Recursion Tree Method

Explanation

- In this method, we draw a recursion tree as we did in our analysis of Merge-Sort algorithm.
- In a **recursion tree**, each node represents the cost of a single subproblem somewhere in the set of recursive function invocations.
- We sum the costs in each level of the tree to obtain a set of per-level costs, and then we sum all the per-level costs to determine the total cost of all levels of the recursion.
- A recurrence tree is best used to generate a **good guess**, which is often verified by the **substitution method**.

Recursion Tree Method

Explanation

- In this method, we draw a recursion tree as we did in our analysis of Merge-Sort algorithm.
- In a **recursion tree**, each node represents the cost of a single subproblem somewhere in the set of recursive function invocations.
- We sum the costs in each level of the tree to obtain a set of per-level costs, and then we sum all the per-level costs to determine the total cost of all levels of the recursion.
- A recurrence tree is best used to generate a **good guess**, which is often verified by the **substitution method**.
- Since we will be using recursion trees to obtain guesses, we can often tolerate a bit of sloppiness since we will later verify the guess anyways.

Example

Recursion Tree Example

- **Recurrence:** $T(n) = 3T(\lfloor \frac{n}{4} \rfloor) + \Theta(n^2)$.

Example

Recursion Tree Example

- **Recurrence:** $T(n) = 3T(\lfloor \frac{n}{4} \rfloor) + \Theta(n^2)$.
- We create a recursion tree for the recurrence $T(n) = 3T(\frac{n}{4}) + c \cdot n^2$.

Example

Recursion Tree Example

- **Recurrence:** $T(n) = 3T(\lfloor \frac{n}{4} \rfloor) + \Theta(n^2)$.
- We create a recursion tree for the recurrence $T(n) = 3T(\frac{n}{4}) + c \cdot n^2$.
- Total cost at level 0: $c \cdot n^2$. Total cost at level 1 is $\frac{3}{16} \cdot c \cdot n^2$.

Example

Recursion Tree Example

- **Recurrence:** $T(n) = 3T(\lfloor \frac{n}{4} \rfloor) + \Theta(n^2)$.
- We create a recursion tree for the recurrence $T(n) = 3T(\frac{n}{4}) + c \cdot n^2$.
- Total cost at level 0: $c \cdot n^2$. Total cost at level 1 is $\frac{3}{16} \cdot c \cdot n^2$.
- Total cost at level i, where $i \in \{0, 1, \ldots, \log_4 n - 1\}$, is $\left(\frac{3}{16}\right)^i cn^2$.

Example

Recursion Tree Example

- **Recurrence:** $T(n) = 3T(\lfloor \frac{n}{4} \rfloor) + \Theta(n^2)$.
- We create a recursion tree for the recurrence $T(n) = 3T(\frac{n}{4}) + c \cdot n^2$.
- Total cost at level 0: $c \cdot n^2$. Total cost at level 1 is $\frac{3}{16} \cdot c \cdot n^2$.
- Total cost at level i, where $i \in \{0, 1, \ldots, \log_4 n - 1\}$, is $\left(\frac{3}{16}\right)^i cn^2$.
- At level $\log_4 n$, we have $n^{\log_4 3}$ leaves, each of which represents $T(1) = \Theta(1)$ cost.

Example

Recursion Tree Example

- **Recurrence:** $T(n) = 3T(\lfloor \frac{n}{4} \rfloor) + \Theta(n^2)$.
- We create a recursion tree for the recurrence $T(n) = 3T(\frac{n}{4}) + c \cdot n^2$.
- Total cost at level 0: $c \cdot n^2$. Total cost at level 1 is $\frac{3}{16} \cdot c \cdot n^2$.
- Total cost at level i, where $i \in \{0, 1, \ldots, \log_4 n - 1\}$, is $\left(\frac{3}{16}\right)^i cn^2$.
- At level $\log_4 n$, we have $n^{\log_4 3}$ leaves, each of which represents $T(1) = \Theta(1)$ cost. Thus,

$$T(n) = \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i \cdot c \cdot n^2 + \Theta(n^{\log_4 3}) \tag{11}$$

$$= \frac{(3/16)^{\log_4 n} - 1}{(3/16) - 1} \cdot c \cdot n^2 + \Theta(n^{\log_4 3}) \tag{12}$$

$$< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i \cdot c \cdot n^2 + \Theta(n^{\log_4 3}) \tag{13}$$

$$= \frac{1}{1 - (3/16)} \cdot c \cdot n^2 + \Theta(n^{\log_4 3}) = O(n^2). \tag{14}$$

Example

Recursion Tree Example

- **Recurrence:** $T(n) = 3T(\lfloor \frac{n}{4} \rfloor) + \Theta(n^2)$.
- We create a recursion tree for the recurrence $T(n) = 3T(\frac{n}{4}) + c \cdot n^2$.
- Total cost at level 0: $c \cdot n^2$. Total cost at level 1 is $\frac{3}{16} \cdot c \cdot n^2$.
- Total cost at level i, where $i \in \{0, 1, \ldots, \log_4 n - 1\}$, is $\left(\frac{3}{16}\right)^i cn^2$.
- At level $\log_4 n$, we have $n^{\log_4 3}$ leaves, each of which represents $T(1) = \Theta(1)$ cost. Thus,

$$T(n) = \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i \cdot c \cdot n^2 + \Theta(n^{\log_4 3}) \tag{11}$$

$$= \frac{(3/16)^{\log_4 n} - 1}{(3/16) - 1} \cdot c \cdot n^2 + \Theta(n^{\log_4 3}) \tag{12}$$

$$< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i \cdot c \cdot n^2 + \Theta(n^{\log_4 3}) \tag{13}$$

$$= \frac{1}{1 - (3/16)} \cdot c \cdot n^2 + \Theta(n^{\log_4 3}) = O(n^2). \tag{14}$$

Verification of the Guess

Self-Exercise

Show that the solution to recurrence $T(n) = 3T(\lfloor \frac{n}{4} \rfloor) + \Theta(n^2)$ is $T(n) = O(n^2)$ by using the substitution method.

Verification of the Guess

Self-Exercise

Show that the solution to recurrence $T(n) = 3T(\lfloor \frac{n}{4} \rfloor) + \Theta(n^2)$ is $T(n) = O(n^2)$ by using the substitution method.

Tight Analysis

- Notice that $T(n) = \Omega(n^2)$ since the recurrence contains the term $\Omega(n^2)$.

Verification of the Guess

Self-Exercise

Show that the solution to recurrence $T(n) = 3T(\lfloor \frac{n}{4} \rfloor) + \Theta(n^2)$ is $T(n) = O(n^2)$ by using the substitution method.

Tight Analysis

- Notice that $T(n) = \Omega(n^2)$ since the recurrence contains the term $\Omega(n^2)$.
- Since $T(n) = \Omega(n^2)$ and $T(n) = O(n^2)$, we have $T(n) = \Theta(n^2)$.

Examples

Several Examples

- $T(n) = T(n/3) + T(2n/3) + O(n)$.

Examples

Several Examples

- $T(n) = T(n/3) + T(2n/3) + O(n)$.
- $T(n) = T(n-1) + T(1) + O(n)$.

Examples

Several Examples

- $T(n) = T(n/3) + T(2n/3) + O(n)$.
- $T(n) = T(n-1) + T(1) + O(n)$.
- $T(n) = T(\frac{n}{a}) + T(\frac{a-1}{a}n) + O(n)$.

Examples

Several Examples

- $T(n) = T(n/3) + T(2n/3) + O(n)$.
- $T(n) = T(n-1) + T(1) + O(n)$.
- $T(n) = T(\frac{n}{a}) + T(\frac{a-1}{a}n) + O(n)$.

Examples

Several Examples

- $T(n) = T(n/3) + T(2n/3) + O(n)$.
- $T(n) = T(n-1) + T(1) + O(n)$.
- $T(n) = T(\frac{n}{a}) + T(\frac{a-1}{a}n) + O(n)$.

Theorem

The running time of the Merge-Sort algorithm is $\Theta(n \cdot \lg n)$ even if the array is not split evenly into two parts at each step of the recursion. The only requirement is that the smaller part is at least a constant fraction of the array.

Master Method

Motivation

- Master method is a *plug and play method* for solving recurrences of the form $T(n) = a \cdot T(n/b) + f(n)$, where $a \geq 1$ and $b > 1$ are constants, and $f(n)$ is an asymptotically positive function.

Master Method

Motivation

- Master method is a *plug and play method* for solving recurrences of the form $T(n) = a \cdot T(n/b) + f(n)$, where $a \geq 1$ and $b > 1$ are constants, and $f(n)$ is an asymptotically positive function.
- This recurrence describes the running-time of an algorithm that divides a problem of size n into a subproblems, each of size n/b, where a and b are positive constants. The a subproblems are solved recursively, each in time $T(n/b)$.

Master Method

Motivation

- Master method is a *plug and play method* for solving recurrences of the form $T(n) = a \cdot T(n/b) + f(n)$, where $a \geq 1$ and $b > 1$ are constants, and $f(n)$ is an asymptotically positive function.
- This recurrence describes the running-time of an algorithm that divides a problem of size n into a subproblems, each of size n/b, where a and b are positive constants. The a subproblems are solved recursively, each in time $T(n/b)$.
- The cost of **dividing** the problem and **combining** the results of the subproblems is described by the function $f(n)$.

Master Method

Motivation

- Master method is a *plug and play method* for solving recurrences of the form $T(n) = a \cdot T(n/b) + f(n)$, where $a \geq 1$ and $b > 1$ are constants, and $f(n)$ is an asymptotically positive function.
- This recurrence describes the running-time of an algorithm that divides a problem of size n into a subproblems, each of size n/b, where a and b are positive constants. The a subproblems are solved recursively, each in time $T(n/b)$.
- The cost of **dividing** the problem and **combining** the results of the subproblems is described by the function $f(n)$.
- The master method provides a Θ-bound for the recurrence if $f(n)$ and $n^{\log_b a}$ are asymptotically **same or polynomially different**.

Master Method

Motivation

- Master method is a *plug and play method* for solving recurrences of the form $T(n) = a \cdot T(n/b) + f(n)$, where $a \geq 1$ and $b > 1$ are constants, and $f(n)$ is an asymptotically positive function.
- This recurrence describes the running-time of an algorithm that divides a problem of size n into a subproblems, each of size n/b, where a and b are positive constants. The a subproblems are solved recursively, each in time $T(n/b)$.
- The cost of **dividing** the problem and **combining** the results of the subproblems is described by the function $f(n)$.
- The master method provides a Θ-bound for the recurrence if $f(n)$ and $n^{\log_b a}$ are asymptotically **same or polynomially different**.
- As a matter of technical correctness, the recurrence isn't well defined because n/b might not be an integer, however, replacing each of the a terms with either $T(\lfloor \frac{n}{b} \rfloor)$ or $T(\lceil \frac{n}{b} \rceil)$ does not affect the asymptotic behavior of the recurrence.

The Master Theorem

Theorem

Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be an asymptotically positive function, and let $T(n)$ be defined on the nonnegative integers by the recurrence $T(n) = a \cdot T(n/b) + f(n)$, where we interpret n/b to mean either $\lfloor \frac{n}{b} \rfloor$ or $\lceil \frac{n}{b} \rceil$. Then, $T(n)$ can be bounded asymptotically as follows:

- If $f(n) = O(n^{\log_b a - \varepsilon})$ for some constant $\varepsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
- If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \log n)$.
- If $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some constant $\varepsilon > 0$, and if $a \cdot f(n/b) \leq c \cdot f(n)$ for some constant $c < 1$ and all sufficiently large n, then $T(n) = \Theta(f(n))$.

Examples

Example 1

$T(n) = 9T(n/3) + n.$

Examples

Example 1

$T(n) = 9T(n/3) + n$.
$f(n) = n = O(n^{2-\varepsilon}) = O(n^{\log_3 9 - \varepsilon})$ for $\varepsilon \leq 1$ and thus, $T(n) = \Theta(n^2)$ by case 1.

Examples

Example 1

$T(n) = 9T(n/3) + n$.
$f(n) = n = O(n^{2-\varepsilon}) = O(n^{\log_3 9 - \varepsilon})$ for $\varepsilon \leq 1$ and thus, $T(n) = \Theta(n^2)$ by case 1.

Example 2

$T(n) = T(2n/3) + 1$.

Examples

Example 1

$T(n) = 9T(n/3) + n$.
$f(n) = n = O(n^{2-\varepsilon}) = O(n^{\log_3 9 - \varepsilon})$ for $\varepsilon \leq 1$ and thus, $T(n) = \Theta(n^2)$ by case 1.

Example 2

$T(n) = T(2n/3) + 1$.
$f(n) = 1 = \Theta(1) = n^{\log_{3/2} 1}$, and thus, $T(n) = \Theta(\log n)$ by case 2.

Examples

Example 1

$T(n) = 9T(n/3) + n$.
$f(n) = n = O(n^{2-\varepsilon}) = O(n^{\log_3 9 - \varepsilon})$ for $\varepsilon \leq 1$ and thus, $T(n) = \Theta(n^2)$ by case 1.

Example 2

$T(n) = T(2n/3) + 1$.
$f(n) = 1 = \Theta(1) = n^{\log_{3/2} 1}$, and thus, $T(n) = \Theta(\log n)$ by case 2.

Example 3

$T(n) = 3T(n/4) + n \log n$.

Examples

Example 1

$T(n) = 9T(n/3) + n$.
$f(n) = n = O(n^{2-\varepsilon}) = O(n^{\log_3 9 - \varepsilon})$ for $\varepsilon \leq 1$ and thus, $T(n) = \Theta(n^2)$ by case 1.

Example 2

$T(n) = T(2n/3) + 1$.
$f(n) = 1 = \Theta(1) = n^{\log_{3/2} 1}$, and thus, $T(n) = \Theta(\log n)$ by case 2.

Example 3

$T(n) = 3T(n/4) + n\log n$.
$f(n) = n\log n = \Omega(n^{\log_4 3} + \varepsilon)$ for $\varepsilon < 0.2$, and $3(n/4)\log(n/4) \leq (3/4)n\log n = cf(n)$ for $c = 3/4$. Thus, case 3 applies, and $T(n) = \Theta(n\log n)$

More Examples

Example 1

$T(n) = 4T(n/2) + n.$

More Examples

Example 1

$T(n) = 4T(n/2) + n$.
$f(n) = n = O(n^{2-\varepsilon}) = O(n^{\log_2 4 - \varepsilon})$ for $\varepsilon \leq 1$. Thus, case 1 applies and $T(n) = \Theta(n^2)$.

More Examples

Example 1

$T(n) = 4T(n/2) + n$.
$f(n) = n = O(n^{2-\varepsilon}) = O(n^{\log_2 4 - \varepsilon})$ for $\varepsilon \leq 1$. Thus, case 1 applies and $T(n) = \Theta(n^2)$.

Example 2

$T(n) = 4T(n/2) + n^2$.

More Examples

Example 1

$T(n) = 4T(n/2) + n$.
$f(n) = n = O(n^{2-\varepsilon}) = O(n^{\log_2 4 - \varepsilon})$ for $\varepsilon \leq 1$. Thus, case 1 applies and $T(n) = \Theta(n^2)$.

Example 2

$T(n) = 4T(n/2) + n^2$.
$f(n) = n^2 = \Theta(n^2) = O(n^{\log_2 4})$. Thus, case 2 applies and $T(n) = \Theta(n^2 \log n)$.

More Examples

Example 1

$T(n) = 4T(n/2) + n$.
$f(n) = n = O(n^{2-\varepsilon}) = O(n^{\log_2 4 - \varepsilon})$ for $\varepsilon \leq 1$. Thus, case 1 applies and $T(n) = \Theta(n^2)$.

Example 2

$T(n) = 4T(n/2) + n^2$.
$f(n) = n^2 = \Theta(n^2) = O(n^{\log_2 4})$. Thus, case 2 applies and $T(n) = \Theta(n^2 \log n)$.

Example 3

$T(n) = 4T(n/2) + n^3$.

More Examples

Example 1

$T(n) = 4T(n/2) + n$.
$f(n) = n = O(n^{2-\varepsilon}) = O(n^{\log_2 4 - \varepsilon})$ for $\varepsilon \leq 1$. Thus, case 1 applies and $T(n) = \Theta(n^2)$.

Example 2

$T(n) = 4T(n/2) + n^2$.
$f(n) = n^2 = \Theta(n^2) = O(n^{\log_2 4})$. Thus, case 2 applies and $T(n) = \Theta(n^2 \log n)$.

Example 3

$T(n) = 4T(n/2) + n^3$.
$f(n) = n^3 = \Theta(n^{2+\varepsilon}) = O(n^{\log_2 4 + \varepsilon})$ for $\varepsilon \leq 1$. In addition to that, we have
$4(\frac{n}{2})^3 = \frac{1}{2}n^3 = cf(n)$ for $c = \frac{1}{2}$. Thus, case 3 applies and $T(n) = \Theta(n^3)$.

Challenging Example

Recurrence

$T(n) = 2T(n/2) + n \log n.$

Challenging Example

Recurrence

$T(n) = 2T(n/2) + n \log n$. $f(n) = n \log n$ is asymptotically larger than $n^{\log_b a} = n$, but it is not **polynomially** larger, and thus we cannot apply Master Theorem.

Challenging Example

Recurrence

$T(n) = 2T(n/2) + n \log n$. $f(n) = n \log n$ is asymptotically larger than $n^{\log_b a} = n$, but it is not **polynomially** larger, and thus we cannot apply Master Theorem.

Remedy

- We can't get a Θ-bound by using the Master Theorem, however, isn't it possible to get some O-bound.

Challenging Example

Recurrence

$T(n) = 2T(n/2) + n \log n$. $f(n) = n \log n$ is asymptotically larger than $n^{\log_b a} = n$, but it is not **polynomially** larger, and thus we cannot apply Master Theorem.

Remedy

- We can't get a Θ-bound by using the Master Theorem, however, isn't it possible to get some O-bound.
- The Θ-bound for $S(n) = 2S(n/2) + n^{1+\varepsilon}$ for some small $\varepsilon > 0$ will give us a O-bound.

Challenging Example

Recurrence

$T(n) = 2T(n/2) + n\log n$. $f(n) = n\log n$ is asymptotically larger than $n^{\log_b a} = n$, but it is not **polynomially** larger, and thus we cannot apply Master Theorem.

Remedy

- We can't get a Θ-bound by using the Master Theorem, however, isn't it possible to get some O-bound.
- The Θ-bound for $S(n) = 2S(n/2) + n^{1+\varepsilon}$ for some small $\varepsilon > 0$ will give us a O-bound.
- Now, $f(n) = n^{1+\varepsilon} = \Omega(n^{\log_2 2+\varepsilon})$. And, $2(\frac{n}{2})^{1+\varepsilon} = cn^{1+\varepsilon}$ for $c = 2^{-\varepsilon} < 1$. Thus, case 3 applies, and $S(n) = \Theta(n^{1+\varepsilon})$. Then, $T(n) = O(n^{1+\varepsilon})$ for any $\varepsilon > 0$.

Challenging Example

Recurrence

$T(n) = 2T(n/2) + n\log n$. $f(n) = n\log n$ is asymptotically larger than $n^{\log_b a} = n$, but it is not **polynomially** larger, and thus we cannot apply Master Theorem.

Remedy

- We can't get a Θ-bound by using the Master Theorem, however, isn't it possible to get some O-bound.
- The Θ-bound for $S(n) = 2S(n/2) + n^{1+\varepsilon}$ for some small $\varepsilon > 0$ will give us a O-bound.
- Now, $f(n) = n^{1+\varepsilon} = \Omega(n^{\log_2 2+\varepsilon})$. And, $2(\frac{n}{2})^{1+\varepsilon} = cn^{1+\varepsilon}$ for $c = 2^{-\varepsilon} < 1$. Thus, case 3 applies, and $S(n) = \Theta(n^{1+\varepsilon})$. Then, $T(n) = O(n^{1+\varepsilon})$ for any $\varepsilon > 0$.
- **Recursion Tree Method:** Cost at level i, for $i \in \{0, 1, \ldots, \log n - 1\}$: $cn\log n - icn$.

Challenging Example

Recurrence

$T(n) = 2T(n/2) + n\log n$. $f(n) = n\log n$ is asymptotically larger than $n^{\log_b a} = n$, but it is not **polynomially** larger, and thus we cannot apply Master Theorem.

Remedy

- We can't get a Θ-bound by using the Master Theorem, however, isn't it possible to get some O-bound.
- The Θ-bound for $S(n) = 2S(n/2) + n^{1+\varepsilon}$ for some small $\varepsilon > 0$ will give us a O-bound.
- Now, $f(n) = n^{1+\varepsilon} = \Omega(n^{\log_2 2+\varepsilon})$. And, $2(\frac{n}{2})^{1+\varepsilon} = cn^{1+\varepsilon}$ for $c = 2^{-\varepsilon} < 1$. Thus, case 3 applies, and $S(n) = \Theta(n^{1+\varepsilon})$. Then, $T(n) = O(n^{1+\varepsilon})$ for any $\varepsilon > 0$.
- **Recursion Tree Method:** Cost at level i, for $i \in \{0, 1, \ldots, \log n - 1\}$: $cn\log n - icn$. And there are $2^{\log n}$ leaves at level $\log n$ of the tree each of which costs $\Theta(1)$.

Challenging Example

Recurrence

$T(n) = 2T(n/2) + n\log n$. $f(n) = n\log n$ is asymptotically larger than $n^{\log_b a} = n$, but it is not **polynomially** larger, and thus we cannot apply Master Theorem.

Remedy

- We can't get a Θ-bound by using the Master Theorem, however, isn't it possible to get some O-bound.
- The Θ-bound for $S(n) = 2S(n/2) + n^{1+\varepsilon}$ for some small $\varepsilon > 0$ will give us a O-bound.
- Now, $f(n) = n^{1+\varepsilon} = \Omega(n^{\log_2 2+\varepsilon})$. And, $2(\frac{n}{2})^{1+\varepsilon} = cn^{1+\varepsilon}$ for $c = 2^{-\varepsilon} < 1$. Thus, case 3 applies, and $S(n) = \Theta(n^{1+\varepsilon})$. Then, $T(n) = O(n^{1+\varepsilon})$ for any $\varepsilon > 0$.
- **Recursion Tree Method:** Cost at level i, for $i \in \{0, 1, \ldots, \log n - 1\}$: $cn\log n - icn$. And there are $2^{\log n}$ leaves at level $\log n$ of the tree each of which costs $\Theta(1)$.
- Thus, the running time is $\sum_{i=0}^{\log n-1} (cn\log n - icn) + \Theta(n) = \frac{1}{2}cn(\log^2 + \log n) + \Theta(n) = \Theta(n \cdot \log^2 n)$.